TechnoSignia Internship — Day 14 Task

SQL Fundamentals

Topics Covered

1. Introduction to SQL
2. CREATE, ALTER, and DROP
3. INSERT, UPDATE, and DELETE
4. SELECT
5. WHERE, ORDER BY, and LIMIT
6. Filtering Records
7. Constraints
8. Primary and Foreign Keys

1. Introduction to SQL

1. SQL stands for Structured Query Language.
2. It is used to create, manage, and manipulate data in relational databases.
3. SQL allows users to store information in tables containing rows and columns.
4. It is used to insert, retrieve, update, and delete database records.
5. MySQL and PostgreSQL are examples of relational database management systems that support SQL.
6. SQL is commonly used in backend development to communicate with databases.

Example:

query.sql

```sql
-- Display all records from the Students table
SELECT * FROM Students;
```

2. CREATE, ALTER, and DROP

1. **CREATE:** Used to create a new database or database object such as a table.
2. **ALTER:** Used to modify the structure of an existing table, such as adding or changing columns.
3. **DROP:** Used to permanently remove a database or table.
4. These commands belong to Data Definition Language (DDL).
5. DROP removes the table structure and its stored data.

Examples:

query.sql

```sql
-- Create a new database
CREATE DATABASE student_management;

-- Select the database
USE student_management;

-- Create a new table
CREATE TABLE Students (
    student_id INT PRIMARY KEY AUTO_INCREMENT,
```

```
    student_name VARCHAR(100) NOT NULL
);

-- Add a new column to the table
ALTER TABLE Students
ADD phone VARCHAR(15);

-- Remove the Students table completely
DROP TABLE Students;
```

3. INSERT, UPDATE, and DELETE

1. **INSERT:** Used to add new records into a table.
2. **UPDATE:** Used to modify existing records.
3. **DELETE:** Used to remove selected records from a table.
4. These commands belong to Data Manipulation Language (DML).
5. The WHERE clause is important when updating or deleting selected records.

Examples:

query.sql

```
-- Insert a new student record
INSERT INTO Students (student_name)
VALUES ('Aarav');

-- Update the student's name
UPDATE Students
SET student_name = 'Rahul'
WHERE student_id = 1;

-- Delete the student with ID 1
DELETE FROM Students
WHERE student_id = 1;
```

4. SELECT

1. SELECT is used to retrieve data from one or more database tables.
2. The asterisk (*) selects all columns from a table.
3. Specific column names can be used to retrieve only required information.
4. SELECT is one of the most frequently used SQL commands.

Examples:

query.sql

```
-- Retrieve all columns and records
SELECT * FROM Students;

-- Retrieve only student names and email addresses
SELECT student_name, email
FROM Students;
```

5. WHERE, ORDER BY, and LIMIT

1. **WHERE:** Filters records based on a specified condition.

2. **ORDER BY:** Sorts records in ascending (ASC) or descending (DESC) order.

3. **LIMIT:** Restricts the number of records returned by a query.

4. These clauses help retrieve specific and organized information.

Examples:

query.sql

```sql
-- Retrieve students whose IDs are greater than 2
SELECT * FROM Students
WHERE student_id > 2;

-- Sort students alphabetically by name
SELECT * FROM Students
ORDER BY student_name ASC;

-- Display only the first 3 records
SELECT * FROM Students
LIMIT 3;
```

6. Filtering Records

1. Filtering means retrieving only those records that satisfy particular conditions.

2. SQL provides comparison operators such as =, >, <, >=, and <=.

3. `AND` is used when all specified conditions must be true.

4. `OR` is used when at least one condition must be true.

5. `LIKE` is used for pattern matching.

6. `IN` checks whether a value matches any value in a specified list.

7. `BETWEEN` retrieves values within a specified range.

8. `IS NULL` checks for missing or NULL values.

Examples:

query.sql

```sql
-- Retrieve students enrolled in Java
SELECT * FROM Students
WHERE course = 'Java';

-- Retrieve students who are older than 20
SELECT * FROM Students
WHERE age > 20;

-- Retrieve students enrolled in Java or Python
SELECT * FROM Students
WHERE course IN ('Java', 'Python');

-- Retrieve students whose names start with A
SELECT * FROM Students
WHERE student_name LIKE 'A%';

-- Retrieve students whose age is between 18 and 22
SELECT * FROM Students
WHERE age BETWEEN 18 AND 22;

-- Retrieve students who have no phone number
SELECT * FROM Students
WHERE phone IS NULL;
```

7. Constraints

1. Constraints are rules applied to table columns to maintain data accuracy and integrity.
2. **NOT NULL:** Prevents a column from storing NULL values.
3. **UNIQUE:** Ensures that values in a column are not duplicated.
4. **PRIMARY KEY:** Uniquely identifies every record in a table.
5. **FOREIGN KEY:** Establishes a relationship between two tables.
6. **CHECK:** Ensures that values satisfy a specified condition.
7. **DEFAULT:** Assigns a predefined value when no value is provided.

Example:

query.sql

```sql
-- Create a table with different constraints
CREATE TABLE ExampleStudents (
    student_id INT PRIMARY KEY AUTO_INCREMENT,
    student_name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE,
    age INT CHECK (age >= 18),
    status VARCHAR(20) DEFAULT 'Active'
);
```

8. Primary and Foreign Keys

1. A **Primary Key** uniquely identifies each record in a database table.
2. A primary key cannot contain duplicate or NULL values.
3. A **Foreign Key** is a column that references a primary or unique key in another table.
4. Foreign keys maintain referential integrity between related tables.
5. One table can contain multiple foreign keys.
6. A relationship can connect students to courses using a separate enrollment table.

Example:

query.sql

```sql
-- Parent table
CREATE TABLE Courses (
    course_id INT PRIMARY KEY AUTO_INCREMENT,
    course_name VARCHAR(100) NOT NULL
);

-- Child table
CREATE TABLE Enrollments (
    enrollment_id INT PRIMARY KEY AUTO_INCREMENT,
    student_id INT NOT NULL,
    course_id INT NOT NULL,

    FOREIGN KEY (course_id)
    REFERENCES Courses(course_id)
);
```

Practical Tasks and Solutions

Task 1 — Create the Required Database and Tables

Create Database

```sql
-- Create the Student Management database
CREATE DATABASE student_management;

-- Select the database for further operations
USE student_management;
```

Create Students Table

```sql
-- Create a table to store student information
CREATE TABLE Students (
    student_id INT PRIMARY KEY AUTO_INCREMENT,
    student_name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    phone VARCHAR(15),
    age INT CHECK (age >= 18)
);
```

Explanation:

1. `student_id` uniquely identifies each student and automatically increases.
2. `student_name` stores the student's name and cannot be empty.
3. `email` stores the student's email and does not allow duplicates.
4. `phone` stores the student's phone number.
5. `age` stores the student's age and must be at least 18.

Create Courses Table

```sql
-- Create a table to store course information
CREATE TABLE Courses (
    course_id INT PRIMARY KEY AUTO_INCREMENT,
    course_name VARCHAR(100) NOT NULL,
    duration_months INT,
    fees DECIMAL(10, 2)
);
```

Explanation:

1. `course_id` uniquely identifies each course.
2. `course_name` stores the course name.
3. `duration_months` stores the course duration.
4. `fees` stores the course fees with decimal precision.

Create Enrollments Table

```sql
-- Create a table to connect students with their courses
CREATE TABLE Enrollments (
    enrollment_id INT PRIMARY KEY AUTO_INCREMENT,
    student_id INT NOT NULL,
    course_id INT NOT NULL,

    -- Connect student_id with the Students table
    FOREIGN KEY (student_id)
    REFERENCES Students(student_id),

    -- Connect course_id with the Courses table
    FOREIGN KEY (course_id)
    REFERENCES Courses(course_id)
);
```

Explanation:

1. `enrollment_id` uniquely identifies each enrollment.
2. `student_id` references the Students table.
3. `course_id` references the Courses table.
4. The foreign keys ensure that an enrollment refers to an existing student and course.

Task 2 — Insert Student and Course Records

Insert Student Records

```sql
-- Insert sample student records
INSERT INTO Students (student_name, email, phone, age)
VALUES
('Aarav Sharma', 'aarav@gmail.com', '9876543210', 21),
('Priya Patil', 'priya@gmail.com', '9876543211', 22),
('Rahul Deshmukh', 'rahul@gmail.com', '9876543212', 20),
('Sneha Joshi', 'sneha@gmail.com', '9876543213', 21),
('Rohan More', 'rohan@gmail.com', '9876543214', 23);
```

Insert Course Records

```sql
-- Insert sample course records
INSERT INTO Courses (course_name, duration_months, fees)
VALUES
('Java', 6, 15000.00),
('Python', 4, 12000.00),
('React.js', 3, 10000.00),
('Web Development', 5, 18000.00);
```

Insert Enrollment Records

query.sql

```sql
-- Enroll students into their respective courses
INSERT INTO Enrollments (student_id, course_id)
VALUES
(1, 1),
(2, 3),
(3, 2),
(4, 4),
(5, 1);
```

Task 3 — Retrieve and Filter Records

Retrieve All Students

query.sql

```sql
-- Display all student records
SELECT * FROM Students;
```

Retrieve All Courses

query.sql

```sql
-- Display all course records
SELECT * FROM Courses;
```

Retrieve Selected Columns

query.sql

```sql
-- Display only student names and email addresses
SELECT student_name, email
FROM Students;
```

Filter Students by Age

query.sql

```sql
-- Display students older than 20
SELECT * FROM Students
WHERE age > 20;
```

Filter Students by Name

query.sql

```sql
-- Display students whose names start with S
SELECT * FROM Students
WHERE student_name LIKE 'S%';
```

Sort Students by Name

query.sql

```sql
-- Display students in alphabetical order
SELECT * FROM Students
ORDER BY student_name ASC;
```

Display Limited Records

query.sql

```sql
-- Display only the first 3 students
SELECT * FROM Students
LIMIT 3;
```

Retrieve Courses Within a Fee Range

query.sql

```sql
-- Display courses with fees between 10000 and 15000
SELECT * FROM Courses
WHERE fees BETWEEN 10000 AND 15000;
```

Retrieve Student and Course Details Using JOIN

query.sql

```sql
-- Combine student and course information
SELECT
    Students.student_name,
    Courses.course_name
FROM Enrollments
JOIN Students
    ON Enrollments.student_id = Students.student_id
JOIN Courses
    ON Enrollments.course_id = Courses.course_id;
```

Explanation:

1. `JOIN` combines records from related tables.
2. The first JOIN connects Enrollments with Students using `student_id`.
3. The second JOIN connects Enrollments with Courses using `course_id`.
4. The result displays each student's name and enrolled course.

Task 4 — Update Student Information

query.sql

```sql
-- Change the phone number of the student with ID 1
UPDATE Students
SET phone = '9999999999'
WHERE student_id = 1;

-- Change the age of the student with ID 2
UPDATE Students
SET age = 23
WHERE student_id = 2;

-- Verify the updated student records
SELECT * FROM Students;
```

Explanation:

1. `UPDATE` modifies existing records.
2. `SET` specifies the new value.
3. `WHERE` identifies which record should be updated.
4. Without WHERE, all records in the table may be updated.

Task 5 — Delete Selected Records

query.sql

```sql
-- Delete the student with ID 5 from Enrollments first
DELETE FROM Enrollments
WHERE student_id = 5;

-- Delete the student with ID 5
DELETE FROM Students
WHERE student_id = 5;

-- Display the remaining student records
SELECT * FROM Students;
```

Explanation:

1. DELETE removes records from a table.
2. The enrollment record is deleted first because it references the student through a foreign key.
3. The student record can then be deleted.
4. WHERE ensures that only the selected student's record is removed.

Task 6 — Practice Additional SQL Queries

1. Count Total Students

query.sql

```sql
-- Count the number of students
SELECT COUNT(*) AS total_students
FROM Students;
```

2. Find the Youngest Student

query.sql

```sql
-- Find the minimum student age
SELECT MIN(age) AS youngest_age
FROM Students;
```

3. Find the Average Student Age

query.sql

```sql
-- Calculate the average age of students
SELECT AVG(age) AS average_age
FROM Students;
```

4. Find the Highest Course Fee

query.sql

```sql
-- Display the highest course fee
SELECT MAX(fees) AS highest_fees
FROM Courses;
```

5. Find Courses With Fees Greater Than 12000

query.sql

```sql
-- Retrieve courses with fees above 12000
SELECT * FROM Courses
WHERE fees > 12000;
```

6. Count Students in Each Course

query.sql

```sql
-- Count the number of students enrolled in each course
SELECT
    Courses.course_name,
    COUNT(Enrollments.student_id) AS total_students
FROM Courses
LEFT JOIN Enrollments
    ON Courses.course_id = Enrollments.course_id
GROUP BY Courses.course_id, Courses.course_name;
```

7. Display Students in Descending Age Order

query.sql

```sql
-- Display students from oldest to youngest
SELECT * FROM Students
ORDER BY age DESC;
```

8. Display Students Whose Age Is 21 or 22

query.sql

```sql
-- Retrieve students aged 21 or 22
SELECT * FROM Students
WHERE age IN (21, 22);
```